

Accelerating an Exact Unknot Recognizer

Cached Cobordisms, Deferred Elimination,
and Visible Connected-Sum Factorization

Technical implementation report

18 September 2026

Abstract

We modify the Python recognizer supplied in the `fast/` directory of `Knots.zip`, rather than replace it with a speculative hierarchy oracle. The complete fallback still computes Khovanov homology over $\mathbb{F}_2$ and has exponential worst-case complexity. The implemented changes are immutable stage-local cobordism caches, algebraic fast paths, scalar bitset elimination at closure, a configurable deferred crossing tail, certified visible connected-sum factorization, and a bounded exact modular Jones obstruction. On fresh-process, same-machine comparisons, Conway recognition improves from 33.94 to 1.95 milliseconds and Kinoshita–Terasaka recognition from 42.44 to 2.16 milliseconds. A 36-crossing rank computation finishes in a median 7.885 seconds, while the supplied backend reaches its 60-second limit; a three-crossing tail reduces the median to 5.528 seconds at higher memory cost. A genuine restricted-class bound is obtained: with largest visible factor size b , the rank backend costs $\text{poly}(n) + n2^{O(b)}$. This gives quasi-polynomial time when $b = O(\log^2 n)$, but does not give a general quasi-polynomial recognizer. The package contains runnable code, an untouched baseline, 41 test methods, complete benchmark inputs, raw observations, and this report.

Contents

1 Outcome and scope	3
2 What the supplied Lackenby sources establish	3
3 Baseline architecture and the central bottleneck	4
3.1 Why boundary width is not enough	4
4 Immutable cobordisms and algebraic fast paths	5
4.1 Unit inversion collapses to the identity map on units	5
4.2 Topology is separated from coefficients	5
5 Deferred elimination and bit-packed closure	6
5.1 Why cancellation may be postponed	6
5.2 Closed differentials are matrices over $\mathbb{F}_2$	6
6 An exact bounded modular Jones obstruction	7
6.1 Frontier dynamic programming	7
6.2 Bounded overhead	8
6.3 A necessary PD-orientation correction	8
7 Visible connected sums and compressed homology	9
7.1 Finding and verifying a splitting sphere	9
7.2 Replayable certificates	9
7.3 The tensor-product formula	9
8 Correctness and complexity of the complete recognizer	10
8.1 Exactness of the final test	10
8.2 The general exponential bound	11
8.3 A restricted quasi-polynomial guarantee	11
9 Validation and reproducible performance	12
9.1 Test coverage	12
9.2 Benchmark protocol	12
9.3 Interpretation, including regressions	14
10 Using and extending the implementation	15
10.1 Commands	15
10.2 API and output contracts	15
10.3 Resource and trust boundaries	16
11 Delivered artifacts and provenance	16

1 Outcome and scope

The delivered `fast/` is a working replacement for the corresponding directory in the supplied archive. It retains the input formats and the standard-library-only runtime. The original implementation is separately preserved under `baseline/fast/`. The comparison is therefore against actual supplied source, not a reconstructed description of it.

There are three distinct outcomes. First, the recognition pipeline becomes much faster on the supplied Alexander-trivial Conway and Kinoshita–Terasaka examples, because a cheap exact obstruction avoids the homology calculation. Second, the homology backend itself becomes faster on several substantial examples, including a previously timed-out case. Third, a factorized representation gives a provable asymptotic improvement on explicitly decomposable diagrams. These are different claims and are measured separately below.

What is not claimed. No algorithm in this package is asserted to recognize all n -crossing knots in $n^{O(\log n)}$ time. There is no implementation of all the multi-surface, Cheeger-region, and hierarchy simplification operations in Lackenby’s talk. The solution does not confuse a modular invariant with a complete unknot test, does not treat a timeout as a negative answer, and does not infer a worst-case theorem from the benchmark results.

Input model. The mathematical input is a spherical, one-component planar diagram (PD) with n crossings. Each crossing is a cyclically counterclockwise row (a, b, c, d) , with a, c on the under-strand and b, d on the over-strand. Each edge label occurs twice. The implementation checks one-component connectivity and the genus-zero rotation system, and normalizes labels. An empty PD denotes one crossing-free circle. Braid and grid inputs are converted to this model; n throughout the complexity analysis denotes the resulting number of crossings, not necessarily the grid size. Label parsing also costs time in the bit length of the supplied labels.

2 What the supplied Lackenby sources establish

The attached slide deck is dated February 2021. Its final flowchart involves hierarchical multi-surfaces, cutting, Cheeger regions, weak reduction, and simplification [1]. The accompanying paper is *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, dated 25 July 2026. It presents an incompressibility algorithm, an unknot-recognition corollary, and certificate results; its introduction leaves speed-up to future work [2].

Section 9 of the paper bounds the number of iterations by

$$L(g + 1)^L, \tag{1}$$

where L bounds hierarchy length and g bounds surface pattern-complexity. That section explicitly separates iteration counting from the costs of steps whose implementation details have not yet been bounded. The later refinements control hierarchy construction for the paper’s certificate arguments. They do not turn the supplied scanning program into a quasi-polynomial algorithm [2].

There is a useful implementation lesson in (1). Its logarithm is $\log L + L \log(g + 1)$. To deduce a quasi-polynomial time bound from a proposed implementation, one would need, for example,

$$L \log(g + 1) = O((\log n)^2)$$

and quasi-polynomial bounds on encoded state size and the work in each iteration. A lexicographically decreasing complexity alone is insufficient: its numerical range can be exponentially large even when its bit length is small. Similarly, polynomial-time verification of a proposed certificate is not a polynomial-time method of finding that certificate.

A replacement hierarchy backend would need concrete encodings of cut 3-manifolds, boundary patterns and multi-surfaces, exact transformation routines preserving their topological meaning, termination potentials, and accounting for compressed-coordinate bit lengths. Supplying placeholder functions for these operations would not improve the delivered recognizer. This implementation instead makes independently justified changes at bottlenecks that are present and executable in the supplied source.

3 Baseline architecture and the central bottleneck

The original pipeline performs decreasing Reidemeister moves of types I and II, a descending-diagram test, and an Alexander-polynomial obstruction. If these do not decide the input, it uses incremental Bar–Natan scanning over $\mathbb{F}_2$, an approach based on delooping and Gaussian elimination of isomorphic summands [3].

At a partial scan boundary, an object consists of a matching m of exposed edge labels and a homological cube degree h . Closed circles are delooped into the two labels 1 and X of the Frobenius algebra

$$A = \mathbb{F}_2[X]/(X^2), \quad \Delta(1) = 1 \otimes X + X \otimes 1, \quad \Delta(X) = X \otimes X.$$

The differential is a sparse directed graph between objects. Each edge is a sum of dotted cobordisms, represented as a set of dot-subsets; symmetric difference implements addition modulo two. A crossing is tensored in, closed circles are delooped, and invertible differential entries are canceled.

The baseline repeatedly constructs the topology of the same compositions and crossing maps. More importantly, a categorical cancellation can create many new edges: an invertible entry with u other incoming edges and v other outgoing edges can require uv Schur-complement updates. It can be expensive to eliminate a penultimate tangle even when its eventual closure has only scalar coefficients. Reducing the number of objects as early as possible is therefore not always the fastest schedule.

3.1 Why boundary width is not enough

Let w be the largest number of exposed endpoints along a chosen crossing order. A small w limits the geometry of individual matchings and morphisms. It does *not* bound the multiplicity of repeated matching objects in the complex. This distinction matters for both optimization and honest complexity claims.

For example, a visible chain of k trefoils has a scan order of bounded boundary width: process the factors consecutively. Its reduced Khovanov rank over $\mathbb{F}_2$ is 3^k , as proved using the tensor-product argument in Section 7. The supplied backend explicitly retains $2 \cdot 3^k$ final unreduced generators. Thus a bound of the form $\text{poly}(n)2^{O(w)}$ for that explicit-object algorithm would be false even on this simple family. Counting possible matchings and forgetting their multiplicities is not a valid state-count argument.

4 Immutable cobordisms and algebraic fast paths

The module `reference_algebra.py` preserves the original surface formulas. The optimized module retains their mathematical representation but makes nonzero morphisms immutable `frozenset` values. A composition cache is keyed by its two coefficient values and its three boundary matchings. A separate topology cache is keyed only by the three matchings. Crossing-map entries are cached by the two matchings, coefficient, smoothing choices, boundary labels, and crossing slots.

4.1 Unit inversion collapses to the identity map on units

For a matching m with r arcs, its ungraded dotted endomorphism algebra is

$$E_m = \mathbb{F}_2[x_1, \dots, x_r] / (x_1^2, \dots, x_r^2). \quad (2)$$

The constant monomial represents the identity cobordism. A basis monomial is indexed by the subset of arcs carrying dots.

Lemma 4.1. *An element of E_m is a unit exactly when its constant coefficient is one. Every unit u satisfies $u^{-1} = u$.*

Proof. Write $u = c + \nu$, where $c \in \mathbb{F}_2$ and every monomial of ν is nonconstant. Each such monomial squares to zero, and characteristic two kills all cross terms in the square of their sum. Hence $\nu^2 = 0$. If $c = 1$, then $(1 + \nu)^2 = 1$, giving the stated inverse. If $c = 0$, the element is square-zero and cannot be invertible. Equivalently, its augmentation in $\mathbb{F}_2$ vanishes. $\square$

The implementation therefore replaces iterative inversion by a constant logical operation: verify that the empty dot-subset is present and return the immutable value itself. Endomorphism multiplication also avoids surface reconstruction. Multiply two basis dot-subsets by taking their union if they are disjoint, and return zero if they overlap; add equal unions by parity. Identity and zero compositions are handled immediately.

4.2 Topology is separated from coefficients

For general $m_0 \rightarrow m_1 \rightarrow m_2$, the connected components, boundary circles and Euler-characteristic bookkeeping of the composite depend only on (m_0, m_1, m_2) . They do not depend on which basis terms carry dots. The optimized implementation compiles this topology once and evaluates dot counts against it. This is particularly useful when Gaussian elimination changes coefficients without changing boundary types.

Proposition 4.2. *The cache, identity, inverse and endomorphism changes preserve the value of every differential coefficient computed by the supplied surface formulas.*

Proof. The compiled topology is the same disjoint-set gluing computation as in the original formula, moved outside the loops over coefficient terms. Every original term still contributes the same dot counts to the same components, followed by the same Frobenius evaluation. Memoization only reuses results with identical complete keys. The fast paths are the zero and identity laws or the multiplication and inverse laws of (2), established above. Immutable coefficient values prevent a later symmetric-difference update from corrupting an earlier cached value. $\square$

There are at most 16,384 entries in each of the five bounded caches: composition plans, compositions, crossing maps, doubled matching circles, and gluing. They are cleared at each crossing insertion. This avoids the original indefinitely growing geometry caches. The bound is on *entry count*, not bytes; a single coefficient or key can still be large. Geometry helpers remain shared with the reference module, so the reference comparison is a differential test, not an independent formal verification of the underlying topology.

5 Deferred elimination and bit-packed closure

5.1 Why cancellation may be postponed

The elimination rule removes an invertible differential block $\phi : B \rightarrow C$ and updates the remaining block by the Schur complement

$$d'_{a,f} = d_{a,f} + d_{B,f}\phi^{-1}d_{a,C} \quad \text{over } \mathbb{F}_2.$$

This is a chain-homotopy equivalence. Tensoring with another crossing complex preserves the equivalence. Consequently, performing a valid cancellation immediately or keeping the uncanceled complex and closing it later gives the same final homology [3].

The new parameter `tail_crossings` = t retains the final t crossing insertions without categorical elimination between them. At the final closure it uses scalar linear algebra. The default $t = 2$ skips only the penultimate categorical elimination. The choice $t = 1$ still performs every nonclosed categorical reduction, while larger values trade additional chain generators for fewer expensive surface compositions.

Proposition 5.1. *For every positive t , every valid full crossing order, and every completed run, the deferred-tail algorithm returns the same homology dimensions as the supplied scan.*

Proof. All crossing tensor products and delooping maps are unchanged. Each intermediate reduction that is performed is a chain-homotopy equivalence. Not performing a reduction leaves the exact preceding complex intact. After every crossing has been inserted, the resulting complex is therefore chain-homotopy equivalent to the full Khovanov complex, regardless of which optional reductions were performed. Its homology is computed exactly by the scalar method below. $\square$

Starting with M objects immediately before a t -crossing tail, a crude bound on the number of objects after expansion is $8^t M$: there are two smoothings per crossing, and at most two newly closed circles, each with two labels. This is not a bound on the ratio to the fully reduced baseline at the same position. The latter may already have canceled most of its objects. Large t can therefore be a poor choice under a memory ceiling even when it saves arithmetic.

5.2 Closed differentials are matrices over $\mathbb{F}_2$

At an empty boundary there is only the empty matching. Every nonzero coefficient between its copies is the scalar one. Let c_h be the number of generators in cube degree h , and let D_h be the matrix of $d : C^h \rightarrow C^{h+1}$. Then

$$\beta_h = c_h - \text{rank } D_h - \text{rank } D_{h-1}. \quad (3)$$

Instead of using categorical Schur updates to diagonalize these matrices, the program stores a column as a Python integer: its j th bit records the j th matrix entry. Gaussian elimination uses highest nonzero bits as pivots and XOR as column addition.

Lemma 5.2. *The bitset elimination computes $\text{rank } D_h$ exactly, and (3) gives the dimension of homology in degree h .*

Proof. Every XOR is an elementary addition of one column to another over $\mathbb{F}_2$, which preserves the column span. A stored pivot column has a largest nonzero row not shared by any earlier reduced pivot. The pivot columns are therefore independent. A reduced zero column lies in their span; a nonzero reduced column adds one new basis vector. Induction over input columns proves that the number of pivots is the matrix rank. Since $d^2 = 0$, $\text{im } D_{h-1} \subseteq \ker D_h$, so rank-nullity yields (3). $\square$

The implementation checks that all closed coefficients are the scalar one, that their degrees are adjacent, and that computed homology dimensions are nonnegative. With `-check-d2`, it also checks $d^2 = 0$ before discarding the closed complex and at nonclosed scan stages. It rejects an attempt to extend a completed closed complex.

For a closed complex with K generators, a conservative dense bound is $O(K^3)$ bit operations for elimination, with word-parallel XOR reducing practical cost. A more informative word model gives the usual cubic term divided by the number of bits processed per word, plus matrix construction and bookkeeping. This is not a new asymptotic matrix-rank algorithm. Its advantages here are avoiding cobordism dispatch and sparse Schur fill, and using optimized integer arithmetic.

Finally, only the numbers β_h are retained. The program does not allocate one indistinguishable object for every surviving homology class. This output compression is separate from, and complementary to, the connected-sum factorization below.

6 An exact bounded modular Jones obstruction

A nontrivial Alexander polynomial proves knottedness, but some important knots have Alexander polynomial one. Computing full Khovanov homology on every such input can be unnecessary. We insert a deterministic, one-sided obstruction based on the Kauffman bracket [4].

Fix

$$p = 1,000,000,007, \quad a = 2 \in \mathbb{F}_p, \quad \delta = -a^2 - a^{-2} = -17/4 \neq 0.$$

For each smoothing state $s \in \{0, 1\}^n$, let $\ell(s)$ be its number of closed circles. With the implementation's cyclic PD and crossing-sign conventions, define

$$J_p(D) = (-a^3)^{-w(D)} \sum_s a^{n-2|s|} \delta^{\ell(s)-1}. \quad (4)$$

This is an exact residue of the writhe-normalized bracket. The crossing-free unknot is assigned value one directly. Equivalently, the frontier scan accumulates $Z = \sum_s a^{n-2|s|} \delta^{\ell(s)}$ and finally multiplies by $\delta^{-1}(-a^3)^{-w(D)}$.

6.1 Frontier dynamic programming

The state table maps boundary matchings to coefficients in $\mathbb{F}_p$. For each crossing, extend each matching through both smoothings. Smoothing zero has weight a , smoothing one has weight a^{-1} , and closing c circles adds δ^c . Contributions to an equal matching are added modulo p ; zero coefficients are deleted. The boundary point set is determined by the processed crossings, independently of smoothing choices.

Lemma 6.1. *After any prefix, each table coefficient equals the sum of all smoothing weights of that prefix that induce its matching, including the factors for every already closed circle.*

Proof. Initially the empty prefix has the empty matching with coefficient one. Every extension chooses exactly one of the next crossing's two smoothings. Gluing determines the new matching and the number of new circles, giving exactly the corresponding multiplicative weight. Equal resulting matchings are collected by addition. Induction proves the claim. At closure the normalization in (4) follows. $\square$

If all coefficients vanish at an intermediate stage, the partial skein value is zero. Every remaining extension is linear, so the final value is also zero and can be reported immediately.

Theorem 6.2 (One-sided soundness). *If a completed modular evaluation has $J_p(D) \neq 1$, then D is not an unknot diagram. If the value is one, or the evaluation exceeds its state cap, no conclusion is drawn.*

Proof. The bracket skein rules with $\delta = -a^2 - a^{-2}$ are invariant under Reidemeister moves II and III. A type-I move multiplies the bracket by $-a^3$ or $-a^{-3}$, exactly canceled by the writhe factor. Thus (4) is an ambient-isotopy invariant with value one on the unknot [4]. Reducing the integer Laurent-polynomial identity modulo p and substituting the unit a preserves it. A different residue therefore contradicts unknottedness. No converse is used. $\square$

This is not a probabilistic identity test. The modulus and evaluation point are fixed; there is no false-positive probability to estimate. Collisions can make a knotted diagram inconclusive, but cannot make an unknot return a value different from one when the implementation is correct.

6.2 Bounded overhead

The default cap is $B = 4096$ states. During insertion, attempting to retain more than B nonzero matchings aborts the filter, not the recognizer. It is acceptable to abort even if later cancellation could shrink the table: this only loses an opportunity for a fast obstruction. There are at most $2nB$ completed transition attempts before normal termination, apart from the single overflowing insertion. A matching and its gluing can be processed in polynomial time in n . Greedy ordering is also polynomial. Thus, with fixed B , this filter has polynomially bounded work and storage and cannot add an exponential preliminary phase. The unlimited standalone filter remains an exact but potentially exponential invariant computation.

6.3 A necessary PD-orientation correction

The supplied crossing-sign routine effectively assumed that the incoming under-strand always occupied slot zero. A legal half-turn of a PD row changes that slot to two. For the Jones normalization, the true oriented writhe must be invariant under this representation change.

Let $u \in \{0, 2\}$ and $o \in \{1, 3\}$ be the incoming under- and over-slots obtained by following the knot. The corrected convention is

$$\varepsilon = +1 \text{ if } o - u \equiv 3 \pmod{4}, \quad \varepsilon = -1 \text{ otherwise.}$$

Adding two to both slots preserves the sign. Reversing the orientation of the whole knot also adds two to both and preserves it. The Alexander matrix now uses the actual incoming under-arc together with

these signs; this preserves the original Fox-row meaning. This correction is essential to safely reuse writhe in a new invariant. It is not presented as evidence that the original recognizer's Alexander verdicts were wrong.

7 Visible connected sums and compressed homology

7.1 Finding and verifying a splitting sphere

Let G be the connected four-valent projection graph embedded in the sphere. Its face cycles are already available from the PD rotation system. Two distinct edges in its embedded dual joining the same two distinct faces form a simple dual two-cycle. The corresponding Jordan curve meets the knot projection in exactly two edge interiors. Lifting it to a splitting sphere gives a visible connected-sum decomposition.

The implementation groups primal edges by the unordered pair of their incident faces. A parallel dual pair supplies a candidate (e, f) . The verifier removes these two primal edges, checks disconnection, and checks that each removed edge crosses between the two sides. The graph is connected and all degrees are even, so it has no bridge. A disconnecting pair is therefore a bond with two connected sides. In a spherical embedding, its dual is a simple cycle. Both sides must contain crossings; they are closed by joining their two dangling endpoints with a fresh edge label and independently validated as one-component spherical PDs.

This construction recognizes *visible* sums, including diagrams with nugatory factors. It need not find a factorization hidden by isotopy, and a leaf is not claimed to be a prime knot.

7.2 Replayable certificates

The decomposition is an iterative tree traversal, not Python recursion. Each internal certificate node records its normalized PD, two removed edges, and two child indices; each leaf records a factor index. The verifier reconstructs every cut and checks the child diagrams, reachability, absence of repeated children, and consecutive factor numbering. This certifies the topological decomposition. It does not certify the subsequent homology arithmetic independently of the rank routine.

There are at most n nonempty leaves and fewer than $2n$ tree nodes. Naively storing each node's diagram takes $O(n^2 \log n)$ bits. With ordinary map operations, face grouping and each cut are linear after normalization; repeated sorted-label normalization gives a conservative polynomial total bound, for example $O(n^2 \log n)$ comparison-model work. No exponentially large prime-decomposition search is present.

7.3 The tensor-product formula

Proposition 7.1. *For visible connected-sum diagrams $D = D_1 \# D_2$,*

$$\widehat{\text{Kh}}(D; \mathbb{F}_2) \cong \widehat{\text{Kh}}(D_1; \mathbb{F}_2) \otimes_{\mathbb{F}_2} \widehat{\text{Kh}}(D_2; \mathbb{F}_2)$$

as homologically graded vector spaces, with cube degrees adding. Consequently the total reduced dimensions multiply.

Proof. Choose the reduction basepoints on the two arcs used for the connected sum. A smoothing of D is a pair of smoothings of D_1, D_2 . The two marked circles are joined to form one marked circle, and

all unmarked circles are unchanged. In the reduced complex the marked circle has its fixed X label. The labels on all other circles identify the reduced generators of D with pairs of reduced generators of its factors.

A differential changes one crossing in exactly one factor. Away from the marked circle its multiplication or comultiplication is unchanged. At the marked circle, multiplication satisfies $X \cdot 1 = X$, $X \cdot X = 0$, and splitting satisfies $\Delta(X) = X \otimes X$, so the same identification also respects those terms. In characteristic two there is no tensor-sign issue. Hence the reduced chain complex itself is the tensor product of the two reduced complexes. Over a field each finite complex splits into its homology with zero differential and contractible two-term pieces. Tensoring this splitting proves the homology formula. A smoothing's cube degree is the sum of its two factor degrees, proving the grading statement. $\square$

The scanner computes unreduced dimensions. The characteristic-two splitting of unreduced into two quantum shifts of reduced Khovanov homology gives

$$\dim_{\mathbb{F}_2} \text{Kh}^h(D) = 2 \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}^h(D), \quad (5)$$

when quantum degree is forgotten [5, Corollary 3.2.C]. Thus every factor's unreduced degree counts are halved, convolved as ordinary integer coefficient polynomials, and doubled at the end. The program checks evenness before halving.

If $P_i(z) = \sum_h \dim \widetilde{\text{Kh}}^h(D_i) z^h$, the returned degree polynomial is $2 \prod_i P_i(z)$. A memo table reuses a rank computation only for exactly equal normalized factor PDs; it does not assume that arbitrary isotopic factors can be cheaply recognized as equal. Even without such reuse, factorwise scanning gives the bound in the next section.

8 Correctness and complexity of the complete recognizer

8.1 Exactness of the final test

Kronheimer and Mrowka prove that reduced Khovanov homology detects the unknot [6]. In particular, a nontrivial knot has reduced rational homology of total rank greater than one. The universal coefficient theorem implies that its total reduced dimension over $\mathbb{F}_2$ is at least the rational rank. For the unknot the reduced dimension over $\mathbb{F}_2$ is one. Therefore

$$D \text{ is an unknot} \iff \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(D) = 1. \quad (6)$$

The code obtains the reduced dimension through (5).

Theorem 8.1 (Correctness, assuming the standard Khovanov construction). *Without resource limits, the delivered algorithm terminates on every valid finite classical knot diagram and returns its correct unknot status. With resource limits, every completed `UNKNOT` or `KNOTTED` verdict has the same mathematical meaning; exhaustion may instead produce `UNKNOWN`.*

Proof. The retained Reidemeister reductions preserve knot type, and a descending diagram is unknotted. A nontrivial Alexander polynomial is a valid obstruction. Section 6 proves the added Jones filter one-sided sound. An inconclusive filter falls through rather than deciding. Sections 4 and 5 prove that the optimized scan preserves the exact final homology. Section 7 proves correctness of optional factorization and recombination. The final test is (6). Each scan has finitely many

crossings and finite chain spaces; each categorical cancellation removes two objects. Hence the unlimited computation terminates. Resource exceptions are caught as `UNKNOWN`, not interpreted as a mathematical verdict. $\square$

This is a mathematical correctness argument for the transformations, not a machine-checked proof of the Python program or a reproof of the deep unknot-detection theorem. The baseline construction, input topology, integer operations and implementation details remain part of the trusted software base. The tests in Section 9 exercise these assumptions in several different ways.

8.2 The general exponential bound

Proposition 8.2. *For normalized n -crossing PD inputs, fixed tail parameter, and no resource ceilings, the delivered complete algorithm admits a bound $2^{O(n)}$ on worst-case time and space. No better general bound is established here.*

Proof. A crossing insertion has two smoothings and at most two newly closed circles. Thus the number of object copies ever created is bounded by a constant to the power n , even without intermediate cancellations. There are at most exponentially many source-target pairs. A morphism between two matchings has at most exponentially many dot-subset basis terms, since the number of boundary circles is $O(n)$. Each gluing plan has polynomial-size topological bookkeeping, and a direct term expansion has $2^{O(n)}$ work. There are at most exponentially many pivots, and at most exponentially many Schur updates per pivot. A product of this fixed number of exponential bounds is still $2^{O(n)}$.

The final bitset matrices have at most exponentially many rows and columns, so polynomial-time linear algebra in their dimension is also $2^{O(n)}$. The bounded Jones filter and the other preliminary operations add only polynomial work in their explicit input size. Factorization performs at most n such scans on smaller inputs. Its output coefficients have $O(n)$ bits, because even the original full cube has only exponentially many generators. Integer convolution and certificate processing are polynomial outside the factor scans. These observations give the stated coarse bound. $\square$

This bound is deliberately coarse; it is not an assertion that Python constants or memory behavior are benign. In particular, a bounded number of cache entries does not make coefficients small, and final output compression alone does not prevent large intermediate matrices.

8.3 A restricted quasi-polynomial guarantee

Theorem 8.3. *Suppose the implemented visible-cut procedure decomposes a diagram into factors with crossing counts $n_1, \dots, n_k$, and put $b = \max_i n_i$. Then its factored rank computation takes*

$$\text{poly}(n) + \sum_{i=1}^k 2^{O(n_i)} \leq \text{poly}(n) + n2^{O(b)}. \quad (7)$$

In particular it is polynomial for $b = O(\log n)$ and quasi-polynomial for $b = O((\log n)^2)$.

Proof. Splitting adds no crossings, so $\sum_i n_i = n$ and $k \leq n$ for nonempty factors. Decomposition and certificate construction are polynomial. Apply Proposition 8.2 to each factor and sum the bounds. Reduced degree polynomials have degree at most n and coefficients of $O(n)$ bits, so their naive convolution adds only polynomial work. Identical-factor memoization can decrease the sum but is not needed for the inequality. Substitute the stated bounds on b to obtain the two special cases. $\square$

For k trefoils, Proposition 7.1 gives reduced rank 3^k . The original explicit-object backend has an $\Omega(3^k)$ output-state lower bound, whereas the delivered factorized backend stores 3^k in $O(k)$ bits and runs in polynomial time on the visible chain diagrams. This is a real asymptotic improvement of the exact rank computation. It is not an exponential-to-polynomial claim for the default recognition pipeline on trefoil sums: their Alexander polynomial already obstructs unknottedness cheaply. Nor does (7) settle general recognition, since a prime or hidden-factor input can have $b = n$.

9 Validation and reproducible performance

9.1 Test coverage

The final release passes 41 `unittest` methods on CPython 3.13.5. The supplied suite contributes 19 methods. Its three changed expectations are narrowly tied to the new filter: a default Conway result now names `modular-jones`, and two tests intended to reach a Khovanov resource ceiling explicitly disable the new Jones shortcut. No test was deleted; the untouched originals remain with the baseline.

The new tests include the following independent and differential checks. A full smoothing-cube Jones evaluator uses its own edge-label disjoint-set implementation, not scan matchings or surface gluing. It agrees on 120 seeded random knots and every supplied example with at most 12 crossings. Further tests cover 60 reduction-invariance cases, 40 random half-turned row representations with Alexander and writhe checks, 30 crossing-order and relabeling cases, 25 third-Reidemeister braid relations, 30 mirror relations, and families of positive and negative curls.

There are 6,000 sampled surface-composition comparisons on all five planar matchings of six endpoints, every unit in a three-arc endomorphism algebra, and crossing-map comparisons for zero through four old boundary labels. These are comparisons against retained original formulas; shared geometry helpers limit their independence.

For homology, 35 additional random knots are checked against the original independent dense reduced cube using each of three tail choices and random full crossing orders, with d^2 checking enabled. The original suite has its own dense-cube checks. Mixed connected sums are compared both to the unfactored scan and to the dense cube, including agreement in every cube degree. Tests replay and tamper with certificates, check reuse of 16 trefoil factors, reject malformed crossing orders, and exercise resource limits. Mocking a Jones value of one on Conway is explicitly tested to ensure that the program still falls through to an exact knotted verdict.

These tests are not exhaustive over all diagrams, and seeded random braids are not a uniform sample of knot types. No Lean formalization or independent large-scale knot-library verification is claimed. Raw test output is in `results/tests.txt`.

9.2 Benchmark protocol

The driver `tools/benchmark.py` launches each observation in a fresh process, sequentially, with the same interpreter and `PYTHONHASHSEED=0`. Inputs are retained verbatim in `results/fixtures.json`. The baseline imports the untouched `baseline/fast/`; the optimized variant imports the replacement `fast/`. The driver compares completed exact ranks in every cube degree, not only their sum, and checks recognition-verdict agreement.

The recorded machine is an x86-64 Linux container reporting an AMD EPYC 9V74 80-Core Processor, using CPython 3.13.5. This does not mean 80 cores were used: each worker is single-process and the trials were run sequentially. No CPU pinning or operating-system noise isolation was

performed. The 146 observations cover 25 fixtures. Ordinary variants have three runs; the censored stress baseline and the $t = 4$ stress variant have one. Reported values are medians unless stated otherwise.

The timer surrounds the API call. It excludes Python startup and initial JSON-to-**Diagram** parsing; the new rank API's internal PD validation is included. Microsecond-scale cases are correspondingly noisy and their ratios should not be overinterpreted. The composition statistic now counts actual optimized composition calls; it is not normalized against the original counter, which counted some calls twice. Every worker has a 60-second cooperative algorithm budget and a 65-second outer process limit. The stress baseline returned UNKNOWN after 60.061 seconds, rather than completing. No exact speedup factor is inferred from a timed-out completion. The original archive also contains Windows timing data; none of the ratios below compare our Linux runs with that different machine's measurements.

Case	Baseline (ms)	Optimized (ms)	Ratio
Conway	33.939	1.946	17.44×
Kinoshita–Terasaka	42.435	2.158	19.67×
Hard 8-crossing unknot	1.649	2.439	0.68×
Figure-eight	0.109	0.120	0.91×
Grid determinant-one knot	0.732	0.714	1.03×
Scrambled grid unknot	0.251	0.275	0.91×
$T(3, 5)$	0.559	0.579	0.97×
Trefoil	0.109	0.095	1.14×
Crossing-free unknot	0.007	0.011	0.68×
40-crossing braid unknot	6.662	6.150	1.08×

Table 1: Full recognition pipeline on every supplied example. Ratios below one are regressions. Medians of three fresh-process API timings; milliseconds exclude Python startup.

Case	Baseline (s)	Optimized (s)	Ratio
Conway, $n = 11$	0.034976	0.010611	3.30×
Kinoshita–Terasaka, $n = 11$	0.036908	0.013587	2.72×
Hard unknot, $n = 8$	0.002146	0.002484	0.86×
$T(3, 5)$, $n = 10$	0.009408	0.009157	1.03×
$T(3, 11)$, $n = 22$	0.048958	0.031593	1.55×
Braid unknot, $n = 40$	0.008593	0.007246	1.19×
Random 3-braid, $n = 40$	0.620738	0.166974	3.72×
Random 4-braid, $n = 41$	0.950309	0.280528	3.39×
Random 5-braid, $n = 36$	> 60 (limited)	7.884998	> 60/7.885

Table 2: Exact rank backend without Reidemeister preprocessing or invariant filters. Medians of three runs, except the one censored baseline stress run. A limited run is not a completed timing.

Trefoil factors	Crossings	Reduced rank	Baseline (s)	Optimized (s)
2	6	9	0.002145	0.001058
4	12	81	0.015667	0.001416
6	18	729	0.194700	0.002015
8	24	6,561	1.347997	0.002148
16	48	43,046,721	not run	0.004677
32	96	1,853,020,188,851,841	not run	0.015000

Table 3: Visible connected sums, rank backend only. The optimized computation scans one normalized trefoil factor and reuses it; all other work, including decomposition and degree convolution, is timed.

Tail t	Runs	Time (s)	Peak generators	Peak RSS (MiB)
2	3	7.884998	74,046	239.5
3	3	5.527760	117,318	304.1
4	1	8.225715	229,818	519.4

Table 4: Tail trade-off on the 36-crossing stress case. Times are medians (one run for $t = 4$); RSS is the maximum process high-water mark across the runs and includes runtime overhead. Every run returns reduced rank 2,949 and the same degree dimensions.

9.3 Interpretation, including regressions

The full pipeline improves Conway by $17.44\times$ and Kinoshita–Terasaka by $19.67\times$ in the retained observations. Both are detected by the modular Jones obstruction, so this is primarily a decision-pipeline gain, not a claim that their homology was computed 20 times faster. The raw rank comparisons show backend gains independently: the random 3-braid and 4-braid cases improve by $3.72\times$ and $3.39\times$, respectively. Their reduced ranks are 321 and 109.

On the 36-crossing stress case the default two-crossing tail returns unreduced rank 5,898, hence reduced rank 2,949, in 7.885 seconds. The baseline is censored at 60 seconds. The ratio of that budget to the optimized median is 7.61; it is a budget-relative comparison, not a measured baseline completion ratio. The three-crossing tail reaches the same degree distribution in 5.528 seconds, with a budget-relative ratio 10.85. The larger tail has 117,318 peak generators instead of 74,046. At $t = 4$ memory and bitset fill increase further and the computation slows again. Thus $t = 3$ is useful for this case, not established as a universally better default.

The hard 8-crossing unknot is a documented regression: raw rank rises from 2.15 to 2.48 milliseconds, and full recognition from 1.65 to 2.44 milliseconds. A Jones value of one supplies no shortcut there; filter and validation overhead plus the deferred expansion dominate a tiny problem. Several other easy examples differ by only microseconds. The improved version is not uniformly faster on every input or under every object cap.

Visible sums display a different effect. Eight trefoil factors improve from 1.348 seconds to 0.002148 seconds, about $628\times$, for exact rank. Thirty-two factors have reduced rank $3^{32} = 1,853,020,188,851,841$, returned in 0.015 seconds without materializing that many objects. The baseline was not run at 16 or 32 factors, and the tables say so.

Large-case cross-check. The three tested tail choices agree in every cube degree. A separate run reduces the same input from 36 to 24 crossings using the retained legal Reidemeister moves, then

recomputes its rank from that different PD. The optimized backend again returns reduced rank 2,949. Its cube-degree counts shift down by six, exactly matching the change from 16 to 10 negative crossings; after the usual homological shift every dimension agrees. This changed diagram takes 25.13 seconds in the validation run, illustrating that fewer crossings do not guarantee a faster scan. The baseline on that changed diagram is also inconclusive at its separate 30-second validation limit. Thus the large rank has multiple internal cross-checks, but is not independently verified by a completed baseline or third-party homology computation. The complete transformed PD, move trace and observations are retained in `results/stress_validation.json`.

10 Using and extending the implementation

10.1 Commands

From the delivered `fast/` directory:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot khovanov examples/kinoshita_terasaka.json
python -m fastunknot jones examples/trefoil.json
python -m unittest discover -s tests -v
```

To force the exact backend rather than allow either polynomial obstruction:

```
python -m fastunknot recognize examples/conway.json \
    --no-alexander --no-jones
```

To tune the tail or impose cooperative limits:

```
python -m fastunknot khovanov examples/conway.json --tail-crossings 3
python -m fastunknot recognize examples/conway.json \
    --seconds 30 --max-objects 100000
```

From the archive root, reproduce the comparisons and regenerate the tables:

```
python tools/benchmark.py --repeats 3 --timeout 60
python tools/report_tables.py
python tools/validate_stress.py
sh docs/build.sh
```

The last step requires a LaTeX installation; the runtime and tests require only Python. The PDF already included in the ZIP requires no build.

10.2 API and output contracts

`recognize` returns `UNKNOT`, `KNOTTED`, or `UNKNOWN`, with evidence and a method name. Its JSON explicitly says `quasipolynomial_guarantee: false`. The added controls are `use_jones`, `jones_max_states`, `decompose`, and `tail_crossings`. The original reduction, descending and Alexander controls remain.

`khovanov_rank` returns total and reduced dimensions, dimensions by original cube degree, statistics, and scan order. It does not perform Reidemeister preprocessing. Cube degrees are not the usual writhe-shifted homological degrees; under simplification, a degree shift can occur. Quantum degrees and integral torsion are not computed. When factorization is used, there is no single global scan order:

the result records an empty global order, individual factor results and local orders, and the splitting certificate. An explicitly supplied order bypasses factorization and must be a full permutation of the crossing indices.

The standalone `jones` command returns an evaluation, not an unknot verdict. A successful command exit does not make a value of one a recognition certificate. The CLI uses exit 2 for invalid input, 3 for resource exhaustion or a capped standalone Jones evaluation, and 4 for internal arithmetic inconsistency.

10.3 Resource and trust boundaries

Time checks are cooperative. A long sort, allocation, polynomial operation or integer operation can overrun a requested deadline before the next check. The object ceiling is a generator count, not a hard byte limit; some expansion bookkeeping is constructed before its size is tested. For strict containment, run the command under a separate process with operating-system memory and wall-time controls. Python’s global caches can be cleared by another concurrent call; this affects reuse, not mathematical values, but per-process workers are preferable for reproducible timings.

The reference surface formulas remain in the release for regression tests. A practical next development step would be profiling-guided elimination scheduling based on measured Schur fill and estimated closure size, rather than a fixed tail. Such a scheduler would still need conservative memory accounting. It is not implemented or used to justify these benchmarks. A truly different general complexity bound would require an additional structural theorem or a different complete backend, not merely adjusting the cache size or declaring scan width logarithmic.

11 Delivered artifacts and provenance

Path	Contents
<code>fast/</code>	Improved package, input examples, original compatibility benchmark, and expanded tests.
<code>baseline/fast/</code>	Untouched supplied implementation, tests, examples, and original historical timing files.
<code>docs/</code>	This article, PDF, generated tables, large-case validation note, and build script.
<code>tools/</code>	Fresh-process benchmark driver and worker, table generator, and stress validation driver.
<code>results/</code>	Complete fixtures, all current raw benchmark observations, validation output, and final test logs.
<code>README.md</code>	Entry-point commands, measured outcomes and limitations.

The supplied license is preserved. The two scholarly sources in the original `docs/` were read as input; third-party paper and slide PDFs are not redistributed in this replacement package. Bibliographic identifiers below refer to the inspected versions. All performance claims in this article refer to the current retained observations, not to the older archive’s benchmark log.

References

- [1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, quasi-polynomial algorithm talk slides, February 2021, 109-page slide deck supplied as `quasipolynomial-talk.pdf`. <https://people.>

maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf.

- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. In particular, Introduction, Section 9, Proposition 9.1, and the refinements in Sections 10–14. <https://arxiv.org/abs/2607.23350v1>.
- [3] D. Bar–Natan, *Fast Khovanov homology computations*, Journal of Knot Theory and Its Ramifications **16**(3) (2007), 243–255. arXiv:math/0606318. <https://www.math.toronto.edu/drorbn/papers/FastKh/>.
- [4] L. H. Kauffman, *State models and the Jones polynomial*, Topology **26**(3) (1987), 395–407. [https://doi.org/10.1016/0040-9383\(87\)90009-7](https://doi.org/10.1016/0040-9383(87)90009-7).
- [5] A. Shumakovitch, *Torsion of the Khovanov homology*, arXiv:math/0405474v2, 19 June 2018; especially Corollary 3.2.C. <https://arxiv.org/abs/math/0405474>.
- [6] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publications Mathématiques de l’IHÉS **113** (2011), 97–208. arXiv:1005.4346. <https://arxiv.org/abs/1005.4346>.